G ACADEMY OF TECHNOLOGY
n Autonomous Institution Approved by UGC/AICTE/GOK, Affiliated to VTU, Belagavi, Karnataka)
Accredited by NAAC with 'A' grade
RR Nagar, Bengaluru – 560 098

Department of Computer Science & Engineering (AI&ML)

OOP Concept with JAVA

IV Semester

Course Code:

BCI24406C

PROJECT TITLE:
Real-Time Stock Trading System

Academic Year 2025-26

Prepared By:
NAME :Vinnavan Newton

USN : 1GA24CI118

Dept. Of Computer

Science

Engineering With

(AI&ML)

StockFlow

Real-Time Stock Trading Simulator

Technologies: Java 21 · Kotlin 2.1.0 · Compose Multiplatform 1.7.3 · Gradle

Table of Contents

Sl No.	Section	Page
1	Introduction	3
2	Objectives	4
3	Technologies Used	5
3.1	Technology Integration Example	6
4	System Architecture	7
4.1	Presentation Layer (Compose Desktop UI)	7
4.2	Application Logic Layer (Backend)	7
4.3	Data Persistence Layer	8
4.4	Request Processing Flow	8
5	OOP Design & Class Structure	9
5.1	Class Diagram	9
5.2–5.6	Class Field References	10
6	Module Description	12
7	OOP Concepts Demonstrated	15
8	Testing	18
9	Future Enhancements	21
10	Conclusion	24
11	References	25
12	Project File Structure	26
13	Complete Class & Method Reference	27
14	Build & Run Configuration Reference	29

1. Introduction

The global financial markets generate billions of transactions daily, and the ability to understand stock trading mechanics — price fluctuation, portfolio management, risk assessment, and profit-and-loss calculation — is a foundational skill for any aspiring software engineer. However, access to live trading platforms for educational and experimental purposes is frequently restricted by regulatory constraints, the risk of real financial loss, and the complexity of brokerage account setup.

StockFlow was conceived as a purpose-built educational simulator that faithfully replicates the core mechanics of a real-time stock exchange within a safe, zero-risk desktop environment. The application models a market of 20 stocks distributed across 6 industry sectors, simulates continuous price fluctuation using a randomised percentage-change algorithm updated every second, and provides a complete trading workflow encompassing purchase, sale, portfolio tracking, profit-and-loss calculation, transaction history, and watchlist management — all backed by persistent file-based storage that preserves user data between sessions.

The project is implemented in two complementary forms. The first is a lightweight Java console and Swing GUI application demonstrating foundational OOP concepts including encapsulation, composition, and multithreading. The second — and primary deliverable — is a modern Compose Multiplatform Desktop application built with Kotlin and JetBrains Compose, featuring an Apple-inspired dark-mode interface with real-time reactive state management, sector-based filtering, and automatic data persistence. Both versions share a common Java backend architecture, demonstrating how the same business logic layer can serve entirely different presentation layers without modification — a practical embodiment of the separation of concerns principle.

2. Objectives

The following formal objectives were defined prior to commencement of development and served as the measurable evaluation criteria against which the completed system was assessed.

- To design and implement a real-time stock market simulator capable of modelling price fluctuations across 20 stocks in 6 industry sectors, updating every second to replicate the dynamic behaviour of a live exchange.
- To build a complete stock trading workflow — buy, sell, portfolio view, and transaction history — applying OOP principles of encapsulation, composition, and abstraction throughout the backend class hierarchy.
- To implement robust portfolio management with weighted average buy price tracking, enabling accurate per-stock and aggregate profit-and-loss computation in real time as market prices change.
- To develop a persistent data storage system that serialises and deserialises the complete user state — balance, portfolio holdings, average buy prices, watchlist, and full transaction history — to the local filesystem, ensuring continuity between application sessions.
- To create a visually polished, responsive desktop graphical user interface using JetBrains Compose Multiplatform, featuring an Apple-inspired dark colour palette, sector-coloured badges, reactive state updates, and intuitive navigation via a tabbed interface.
- To organise the codebase around the single-responsibility principle, with each Java class encapsulating a single domain concept and the Kotlin UI layer consuming backend services through clean, well-defined public APIs.
- To provide a secondary implementation using Java Swing, demonstrating how the same backend model layer can power fundamentally different GUI frameworks — thereby illustrating the practical value of model-view separation.
- To implement a sector-based filtering system and a user-managed watchlist, demonstrating the use of the Java Collections Framework (HashMap, ArrayList) for efficient data organisation and retrieval.
- To produce a formally documented technical report with illustrative code examples, class diagrams, and output screenshots, communicating the system's design rationale and implementation detail to an academic audience.

3. Technologies Used

The technology stack for StockFlow was selected on three principal criteria: direct relevance to the OOP with Java curriculum, suitability for building reactive desktop applications, and quality of ecosystem support encompassing documentation, tooling, and community.

Layer	Technology	Version	Role
Backend Language	Java	JDK 21	Primary language for all domain model classes (Stock, User, Market, Transaction, DataStore). Chosen for curriculum alignment and strong OOP support.
UI Language	Kotlin	2.1.0	Used exclusively for the Compose Desktop UI layer (Main.kt). Kotlin's concise syntax, null safety, and first-class Compose support make it ideal for reactive UI development.
UI Framework	Compose Multiplatform	1.7.3	JetBrains' declarative UI toolkit for desktop applications. Provides reactive state management, composable functions, and Material Design components.
Legacy UI Framework	Java Swing	JDK built-in	Used in the initial prototype (GuiMain.java) to demonstrate Java-native GUI construction with JFrame, JTable, and Timer.
Concurrency	ScheduledExecutorService	JDK built-in	Manages the real-time price update loop, scheduling stock price recalculations every 1-2 seconds on a dedicated thread pool.
Collections	Java Collections Framework	JDK built-in	HashMap<String,Integer> for portfolio, HashMap<String,Double> for average prices, ArrayList<Stock> for market listings, ArrayList<Transaction> for history.
Data Persistence	File I/O (PrintWriter / BufferedReader)	JDK built-in	Custom text-based serialisation format saves and loads user state from ~/.stockflow/userdata.txt.
Build System	Gradle (Kotlin DSL)	8.x (Wrapper)	Manages dependency resolution, compilation, and run configuration via build.gradle.kts. The Gradle Wrapper eliminates manual installation requirements.
Version Control	Git / GitHub	2.x	Distributed version control for source code management, branching, and development history.

3.1 Technology Integration Example

The following build.gradle.kts file illustrates how the Kotlin, Compose, and Java toolchain are wired together in the project's build configuration:

```
// build.gradle.kts — StockFlow Build Configuration
plugins {
    kotlin("jvm") version "2.1.0"
    id("org.jetbrains.compose") version "1.7.3"
    id("org.jetbrains.kotlin.plugin.compose") version "2.1.0"
}

kotlin { jvmToolchain(21) }

group = "com.stock"
version = "1.0-SNAPSHOT"

repositories {
    mavenCentral()
    maven("https://maven.pkg.jetbrains.space/public/p/compose/dev")
    google()
}

dependencies {
    implementation(compose.desktop.currentOs)
}

compose.desktop {
    application { mainClass = "com.stock.ui.MainKt" }
}
```

4. System Architecture

StockFlow is structured around a two-tier desktop application architecture in which the backend model layer (Java) and the presentation layer (Kotlin Compose) are logically and physically separated into distinct source directories and packages. The backend classes reside in com.stock.backend and contain zero UI dependencies; the UI layer in com.stock.ui consumes backend services through their public APIs.

4.1 Presentation Layer (Compose Desktop UI)

The Compose Desktop UI (Main.kt) provides a complete trading interface organised into four navigable tabs — Market, Watchlist, History, and Settings — along with a persistent header displaying real-time cash balance, net worth, and total profit/loss. The UI is built entirely from composable functions, each responsible for a single visual component: HeaderBar, TabRow, SectorFilter, MarketList, WatchlistView, TransactionHistory, TradePanel, PortfolioPanel, and SettingsView. State is managed reactively using Compose's mutableStateOf primitives, ensuring the UI recomposes automatically whenever backend data changes.

4.2 Application Logic Layer (Backend)

The backend comprises five Java classes, each encapsulating a single domain responsibility:

Class	Responsibility
Stock	Models a single stock instrument with symbol, name, sector, price, open price, day high/low, and a randomised price update algorithm.
Market	Manages the collection of 20 stocks, initialises them across 6 sectors, runs the ScheduledExecutorService simulation loop, and provides lookup methods.
User	Manages user balance, portfolio holdings (HashMap<String,Integer>), average buy prices, transaction history, watchlist, and computes net worth and P&L.
Transaction	Immutable record of a single trade event (type, symbol, quantity, price per share, total amount, timestamp).
DataStore	Static utility class handling serialisation and deserialisation of User state to the filesystem (~/.stockflow/userdata.txt).

4.3 Data Persistence Layer (File-Based Storage)

User state is persisted to a plain-text file at ~/.stockflow/userdata.txt using a custom key-value format. The DataStore.save() method writes the current balance, initial balance, portfolio map, average buy prices, watchlist, and complete transaction history. The DataStore.load() method parses this file on application startup and reconstructs the User object, enabling full session continuity. If no save file exists, a fresh User with a $1,000,000 starting balance is created.

4.4 Request Processing Flow

The following sequence describes the complete processing path for a typical stock purchase operation:

- The user selects a stock from the MarketList composable. The selectedSymbol state variable is updated and the TradePanel composable displays the stock's details, sector, current price, and day high/low range.
- The user enters a quantity in the TradePanel's OutlinedTextField and clicks the Buy button.
- The onBuy lambda retrieves the Stock object from Market.getStock(selectedSymbol) and parses the quantity from the text field.
- The lambda calls user.buyStock(stock, quantity), which validates that the user's balance is sufficient to cover stock.getPrice() * quantity.
- If the balance check passes, the User object deducts the cost from balance, updates the portfolio map (incrementing shares or creating a new entry), recalculates the weighted average buy price, and appends a new Transaction record to the transactions list.
- The sync() callback is invoked, which calls DataStore.save(user) to persist the updated state to disk and refreshes all Compose state variables, triggering automatic UI recomposition.
- The TradePanel displays a success message: "Bought [qty] [symbol]". If the balance was insufficient, the message reads "Insufficient funds" and no state mutation occurs.

Architectural Note: The market simulation runs on a separate thread via ScheduledExecutorService, updating all stock prices every second. The onUpdateCallback Runnable bridges the Java thread into the Compose state system, triggering UI recomposition on each tick without manual polling.

5. OOP Design & Class Structure

The StockFlow class hierarchy was designed in strict adherence to object-oriented principles — encapsulation, single responsibility, and composition — ensuring that each class models a single domain concept with well-defined boundaries and minimal coupling.

5.1 Class Diagram

```
┌──────────────────────────┐
│         Market           │
├──────────────────────────┤
│ - stocks: List<Stock>    │
│ - scheduler: Scheduled.. │
│ - onUpdateCallback: Run. │
├──────────────────────────┤
│ + startSimulation()      │
│ + stopSimulation()       │
│ + getStock(sym): Stock   │
│ + getStocks(): List      │
│ + getSectors(): List     │
└──────────────────────────┘
           │ contains 1..*
           ▼
┌──────────────────────────┐
│          Stock           │
├──────────────────────────┤
│ - symbol: String         │
│ - price: double          │
│ - dayHigh/dayLow: double │
│ - openPrice: double      │
├──────────────────────────┤
│ + updatePrice(): void    │
│ + getChangePercent(): dbl│
└──────────────────────────┘

┌──────────────────────────┐          ┌──────────────────────────┐
│          User            │          │       Transaction        │
├──────────────────────────┤          ├──────────────────────────┤
│ - balance: double        │          │ - type: Type (BUY/SELL)  │
│ - portfolio: Map<S,I>    │  1..* →  │ - symbol: String         │
│ - avgBuyPrice: Map<S,D>  │          │ - quantity: int          │
│ - transactions: List<Tx> │          │ - pricePerShare: double  │
│ - watchlist: List<String>│          │ - timestamp: long        │
├──────────────────────────┤          └──────────────────────────┘
│ + buyStock(Stock,int)    │
│ + sellStock(Stock,int)   │
│ + getNetWorth(Market)    │
│ + getTotalProfitLoss()   │
└──────────────────────────┘
           │ saved/loaded by
           ▼
┌──────────────────────────┐
│        DataStore         │
├──────────────────────────┤
│ + save(User): void       │
│ + load(double): User     │
│ + reset(): void          │
└──────────────────────────┘
```

5.2 Stock Class — Field Reference

Field	Data Type	Access	Description
symbol	String	private, getter	Ticker symbol. Example: "AAPL"

Field	Data Type	Access	Description
name	String	private, getter	Full company name. Example: "Apple Inc."
sector	String	private, getter	Industry classification. Example: "Technology"
price	double	private, getter	Current market price in USD. Example: 189.50
previousPrice	double	private, getter	Price at previous tick. Used for change calculation.
openPrice	double	private, getter	Price at simulation start. Reference for daily change %.
dayHigh	double	private, getter	Highest price reached in current session.
dayLow	double	private, getter	Lowest price reached in current session.
random	Random	private	java.util.Random instance for price fluctuation generation.

5.3 User Class — Field Reference

Field	Data Type	Access	Description
balance	double	private, getter	Current cash balance in USD. Example: 950000.00
initialBalance	double	private, getter	Starting balance at account creation. Example: 1000000.00
portfolio	Map<String,Integer>	private, getter	Stock holdings: symbol to share count. Example: {"AAPL": 50}
avgBuyPrice	Map<String,Double>	private, getter	Weighted average buy price per stock.
transactions	List<Transaction>	private, getter	Complete chronological trade history.
watchlist	List<String>	private, getter	User-curated list of tracked symbols.

5.4 Transaction Class — Field Reference

Field	Data Type	Access	Description
type	Transaction.Type	private, getter	Enum value: BUY or SELL.
symbol	String	private, getter	Stock symbol traded. Example: "GOOG"
quantity	int	private, getter	Number of shares traded. Example: 10
pricePerShare	double	private, getter	Price at time of trade. Example: 2820.00
totalAmount	double	private, getter	Computed: pricePerShare * quantity.
timestamp	long	private, getter	Unix epoch milliseconds of trade execution.

5.5 Market Class — Field Reference

Field	Data Type	Access	Description
stocks	List<Stock>	private, getter	Collection of 20 initialised Stock objects across 6 sectors.
scheduler	ScheduledExecutorService	private	Single-thread executor running the price update loop every 1 second.
onUpdateCallback	Runnable	private, setter	Callback invoked after each price tick to bridge into Compose state.

5.6 DataStore Class — Field Reference

Field	Data Type	Access	Description
DIR	String	private static final	~/.stockflow — persistence directory path.
FILE	String	private static final	~/.stockflow/userdata.txt — data file path.

6. Module Description

The StockFlow application is decomposed into six principal functional modules, each encapsulating a distinct domain of system responsibility. This architecture adheres to the single-responsibility principle, reducing inter-component coupling and enabling each module to be independently understood, tested, and extended.

6.1 Market Simulation Module

The Market Simulation Module governs the creation, initialisation, and continuous price update of all stock instruments in the system. On construction, the Market class initialises 20 Stock objects spanning 6 industry sectors (Technology, Automotive, Finance, Healthcare, Consumer, Fintech). The startSimulation() method launches a ScheduledExecutorService that invokes updatePrice() on every stock at a fixed rate of once per second, then fires the registered onUpdateCallback to propagate changes to the UI layer.

```
// Market.java — Simulation Loop
public void startSimulation() {
    scheduler.scheduleAtFixedRate(() -> {
        for (Stock stock : stocks) {
            stock.updatePrice();
        }
        if (onUpdateCallback != null) {
            onUpdateCallback.run();
        }
    }, 0, 1, TimeUnit.SECONDS);
}
```

6.2 Stock Price Engine — Stock.java

Each Stock object encapsulates a randomised price fluctuation algorithm that simulates realistic market behaviour. On each tick, the updatePrice() method generates a random percentage change between -2% and +2%, applies it to the current price, enforces a minimum floor of $0.01, and updates the day high and day low tracking fields.

```
// Stock.java — Price Update Algorithm
public void updatePrice() {
    this.previousPrice = this.price;
    double changePercent = (random.nextDouble() * 0.04) - 0.02;
    this.price += this.price * changePercent;
    if (this.price < 0.01)
        this.price = 0.01;
    if (this.price > this.dayHigh)
        this.dayHigh = this.price;
    if (this.price < this.dayLow)
        this.dayLow = this.price;
}
```

6.3 Trading Module

The Trading Module implements the buy and sell operations within the User class. The buyStock() method validates the user's balance against the total purchase cost, deducts the cost, updates the portfolio map, recalculates the weighted average buy price using the formula (existingCost + newCost) / (existingQty + newQty), and records a Transaction. The sellStock() method validates that

the user holds sufficient shares, credits the sale revenue to the balance, adjusts the portfolio map (removing the entry entirely if all shares are sold), and records the transaction.

```
// User.java — Buy Stock with Weighted Average Price Tracking
public boolean buyStock(Stock stock, int quantity) {
    double cost = stock.getPrice() * quantity;
    if (cost > balance)
        return false;
    balance -= cost;

    int existingQty = portfolio.getOrDefault(stock.getSymbol(), 0);
    double existingCost = avgBuyPrice.getOrDefault(stock.getSymbol(), 0.0) *
existingQty;
    double newTotalCost = existingCost + cost;
    int newTotalQty = existingQty + quantity;
    avgBuyPrice.put(stock.getSymbol(), newTotalCost / newTotalQty);

    portfolio.put(stock.getSymbol(), newTotalQty);
    transactions.add(new Transaction(Transaction.Type.BUY,
                     stock.getSymbol(), quantity, stock.getPrice()));
    return true;
}
```

6.4 Portfolio & P&L Calculation

The User class exposes two real-time financial computation methods. getNetWorth(Market) iterates over all portfolio entries, retrieves the current market price of each held stock, and sums the total holdings value plus the cash balance. getTotalProfitLoss(Market) returns the difference between the current net worth and the initial balance, providing a single aggregate measure of trading performance.

```
// User.java — Net Worth Computation
public double getNetWorth(Market market) {
    double worth = balance;
    for (Map.Entry<String, Integer> entry : portfolio.entrySet()) {
        Stock stock = market.getStock(entry.getKey());
        if (stock != null)
            worth += stock.getPrice() * entry.getValue();
    }
    return worth;
}

public double getTotalProfitLoss(Market market) {
    return getNetWorth(market) - initialBalance;
}
```

6.5 Watchlist Module

The Watchlist Module provides a simple toggle-based mechanism for users to track stocks of interest without purchasing them. The toggleWatchlist(String symbol) method adds the symbol to the watchlist ArrayList if absent, or removes it if already present. The Compose UI renders the watchlist as a dedicated tab with real-time price updates and one-click removal.

```
// User.java — Watchlist Toggle
public void toggleWatchlist(String symbol) {
    if (watchlist.contains(symbol))
        watchlist.remove(symbol);
```

```
    else
        watchlist.add(symbol);
}
```

6.6 Data Persistence Module — DataStore.java

The DataStore class provides three static methods — save(User), load(double), and reset() — that manage the complete lifecycle of user data persistence. The serialisation format uses a line-based key-value structure, with complex data types (maps, lists, transactions) encoded using delimiter-separated values. Transactions are serialised as pipe-delimited records prefixed with TX=.

```
// DataStore.java — Save Format
BALANCE=950000.0
INITIAL_BALANCE=1000000.0
PORTFOLIO=AAPL:50,TSLA:20
AVG_PRICES=AAPL:189.5,TSLA:245.0
WATCHLIST=NVDA,META,GOOG
TX=BUY|AAPL|50|189.5|1715000000000
TX=BUY|TSLA|20|245.0|1715000060000
```

7. OOP Concepts Demonstrated

The StockFlow project was designed to serve as a comprehensive practical demonstration of the core object-oriented programming concepts covered in the IV Semester OOP with Java curriculum. Each concept is applied in a meaningful, non-trivial manner throughout the codebase.

7.1 Encapsulation

Every domain class in the project enforces strict encapsulation. All instance fields are declared private, with controlled access provided exclusively through public getter methods. Mutation of internal state occurs only through well-defined public methods (buyStock, sellStock, updatePrice, toggleWatchlist) that enforce business rules before modifying state.

```
// Stock.java — Encapsulation: private fields, public getters
public class Stock {
    private String symbol;    // Not accessible directly
    private double price;

    public String getSymbol() { return symbol; }   // Controlled read
    public double getPrice()  { return price;  }

    public void updatePrice() {                     // Controlled mutation
        this.previousPrice = this.price;
        double changePercent = (random.nextDouble() * 0.04) - 0.02;
        this.price += this.price * changePercent;
        if (this.price < 0.01) this.price = 0.01;
    }
}
```

7.2 Abstraction

The User class abstracts the complexity of portfolio management, weighted average price calculation, and net worth computation behind simple, high-level method signatures. The caller of buyStock(stock, quantity) is not required to understand the internal mechanics of cost validation, map updates, or average price recalculation — the method either succeeds and returns true, or fails and returns false, with all internal bookkeeping handled transparently.

Similarly, the DataStore class abstracts the entire persistence mechanism behind three static methods:

```
DataStore.save(user);                  // Serialises all user state to disk
User user = DataStore.load(1000000.0); // Reconstructs complete user from file
DataStore.reset();                     // Deletes saved data
```

7.3 Composition

The Market class demonstrates composition by containing a List<Stock> of 20 individually instantiated Stock objects. The User class composes its transaction history as a List<Transaction>. Neither Market nor User inherits from Stock or Transaction; instead, they contain and manage instances of these classes, reflecting a has-a relationship rather than an is-a relationship.

```
// Market.java — Composition: Market HAS-A collection of Stock objects
public class Market {
    private List<Stock> stocks;  // Market contains Stock objects
    public Market() { stocks = new ArrayList<>(); initializeStocks(); }
}

// User.java — Composition: User HAS-A list of Transaction objects
public class User {
    private List<Transaction> transactions;
    private Map<String, Integer> portfolio;
}
```

7.4 Polymorphism

The Transaction class uses an enum Type { BUY, SELL } to represent the dual nature of trade operations. The same Transaction container represents both buy and sell operations, and the UI renders them differently based on the type field — green badges for BUY, red badges for SELL.

```
// Transaction.java — Enum-based Type Polymorphism
public class Transaction {
    public enum Type { BUY, SELL }
    private Type type;
    // Same object structure serves both BUY and SELL semantics
    // UI differentiates: val color = if (type == BUY) Green else Red
}
```

7.5 Enum Types

The Transaction.Type enum (BUY, SELL) is a first-class Java enum type used throughout the application for type-safe transaction classification. Enums prevent invalid transaction types at compile time, eliminate magic string comparisons, and integrate cleanly with serialisation (Transaction.Type.valueOf(parts[0])) and when expressions in the Kotlin UI.

7.6 Collection Framework Usage

Collection	Class	Purpose	Key Operations
ArrayList<Stock>	Market	Stores 20 initialised stock objects	Iteration, lookup by symbol
HashMap<String,Integer>	User	Portfolio: symbol to share count	getOrDefault, put, remove, containsKey
HashMap<String,Double>	User	Average buy prices: symbol to price	getOrDefault, put, remove
ArrayList<Transaction>	User	Complete trade history	add, toList, reversed (Kotlin)
ArrayList<String>	User	Watchlist: tracked symbols	add, remove, contains
ArrayList<String>	Market	Sector list (dynamic extraction)	add, contains, iteration

8. Testing

A structured, multi-category testing programme was conducted throughout the development lifecycle. All tests were executed by running the application, performing the described operations, and observing the actual results in the UI and the persistence file.

8.1 Functional Test Cases

Test Case	Input / Condition	Expected	Actual	Result
Application launches with default balance	First run, no save file	Balance: $1,000,000.00	Balance: $1,000,000.00	PASS
Market displays all 20 stocks	View Market tab	20 stocks across 6 sectors	20 stocks displayed	PASS
Buy stock with sufficient balance	Select AAPL, qty: 50, click Buy	Balance reduced, portfolio updated	Balance reduced, AAPL: 50 shares	PASS
Buy stock with insufficient balance	Buy 10000 shares of GOOG	Insufficient funds	Insufficient funds	PASS
Sell stock successfully	Sell 10 AAPL (own 50)	Balance increased, portfolio: 40	Balance increased, AAPL: 40	PASS
Sell more shares than owned	Sell 100 AAPL (own 50)	Not enough shares	Not enough shares	PASS
Sell all shares removes from portfolio	Sell all 50 AAPL	AAPL removed from portfolio	AAPL removed	PASS
Transaction history records buy	Buy 10 TSLA	BUY TSLA x10 entry appears	Entry appears with correct details	PASS
Sector filter — Technology	Select Technology filter	Only tech stocks shown	6 tech stocks displayed	PASS
Sector filter — All	Select All filter	All 20 stocks shown	20 stocks displayed	PASS
Stock prices update every second	Observe Market tab for 10 seconds	Prices fluctuate continuously	Prices update every ~1 second	PASS

8.2 Boundary Test Cases

Boundary Test	Input / Condition	Expected	Actual	Result
Buy with quantity 0	Enter 0 in quantity field	No transaction processed	No transaction	PASS
Buy with empty quantity field	Leave quantity blank, click Buy	No crash, no transaction	Graceful no-op	PASS
Non-numeric quantity input	Enter abc in quantity field	Input rejected	Characters not accepted	PASS
Negative quantity input	Enter -5 in quantity field	Input rejected	Minus sign not accepted	PASS
Net worth with empty portfolio	No stocks owned	Net worth equals cash balance	Net worth = balance	PASS

Boundary Test	Input / Condition	Expected	Actual	Result
P&L with zero trades	No trades executed	P&L = $0.00	P&L = +$0.00	PASS
Watchlist add and remove	Star NVDA, then star again	Added then removed	Toggle works correctly	PASS

8.3 Data Persistence Tests

Persistence Test	Procedure	Expected	Actual	Result
State saved on trade	Buy 10 AAPL, check ~/.stockflow/userdata.txt	File contains updated balance and portfolio	File correctly updated	PASS
State restored on restart	Close and reopen application	Previous balance, portfolio, history restored	All data restored correctly	PASS
Watchlist persisted	Add GOOG to watchlist, restart app	GOOG appears in Watchlist tab	GOOG present after restart	PASS
Transaction history persisted	Execute 3 trades, restart app	All 3 transactions present in History	All 3 transactions restored	PASS
Reset clears all data	Click RESET ALL DATA in Settings	Balance reset, portfolio/history cleared	All data cleared, fresh state	PASS
Missing save file handled	Delete userdata.txt, launch app	Fresh user with default balance	Fresh user created	PASS
Corrupted save file handled	Corrupt userdata.txt, launch app	Application starts with default balance	Default balance loaded	PASS

9. Future Enhancements

The following enhancements represent a structured and prioritised roadmap for evolving StockFlow from its current status as an academic project prototype into a feature-complete trading simulation platform.

9.1 Real-Time API Integration

Replacing the randomised price simulation with data from a live financial API — such as the Alpha Vantage REST API or the Finnhub WebSocket feed — would enable the application to display actual market prices, providing a more realistic and educationally valuable trading experience. The Stock class would be refactored to accept external price updates rather than generating them internally, preserving the existing encapsulation model whilst switching the data source.

9.2 SQLite / PostgreSQL Database Backend

Replacing the plain-text file persistence layer with an embedded SQLite database (via JDBC) would provide structured schema enforcement, indexed queries for efficient transaction history retrieval, and ACID-compliant transaction support. A normalised schema with users, portfolio_holdings, and transactions tables would eliminate the fragility of the current delimiter-based serialisation format and support future multi-user scenarios.

9.3 Candlestick Chart Visualisation

Integrating a charting library into the Compose UI would enable the application to render real-time candlestick charts for each stock, displaying open, high, low, and close prices over configurable time intervals. The existing Stock class already tracks openPrice, dayHigh, dayLow, and price — the data required to construct candlestick data points is already available with minimal additional recording.

9.4 Multi-User Support with Authentication

Extending the DataStore to support multiple named user profiles, each with independent balance, portfolio, and transaction history, would transform StockFlow into a multi-player trading competition platform. A login screen composable would allow users to select or create profiles, with each profile stored in a separate file or database table.

9.5 Android Mobile Port

Because the backend classes are pure Java with zero desktop dependencies, the entire com.stock.backend package could be reused in an Android application with a Jetpack Compose UI layer replacing the desktop Compose layer. The shared backend model would require no modification, demonstrating true platform-independent code reuse through proper separation of concerns.

9.6 Automated Trading Strategies (Paper Trading Bots)

Implementing configurable automated trading strategies — such as moving average crossover, momentum-based buying, or simple buy-low-sell-high algorithms — would enable users to test and compare algorithmic trading approaches in the simulated environment. A TradingStrategy interface with different implementations would demonstrate the Strategy design pattern.

9.7 Export Portfolio to CSV / PDF

Adding the ability to export the current portfolio snapshot and complete transaction history to CSV or PDF format would provide users with a tangible record of their simulated trading performance. The Transaction class already contains all the fields required for a complete trade log export.

10. Conclusion

StockFlow represents a technically rigorous and comprehensively documented implementation of a real-time stock trading simulator, developed as a practical demonstration of object-oriented programming principles in Java. The project has fulfilled all nine of its formally stated objectives, delivering a fully operational desktop application that demonstrates command of encapsulation, composition, abstraction, the Java Collections Framework, multithreaded programming, file-based persistence, and modern reactive UI development with Compose Multiplatform.

The system's backend architecture — comprising five cleanly separated Java classes, each encapsulating a single domain responsibility — provides a textbook example of the single-responsibility principle and composition-over-inheritance design philosophy. The dual-GUI implementation (Java Swing and Compose Desktop) further validates the practical benefit of strict model-view separation: the identical backend code powers two entirely different user interfaces without modification.

The market simulation engine faithfully reproduces the dynamic behaviour of a live exchange, with 20 stocks across 6 sectors updating every second, day high/low tracking, percentage change computation, and sector-based filtering. The trading module implements weighted average buy price tracking, enabling accurate per-stock and aggregate P&L computation — a feature commonly found only in production brokerage platforms.

The acknowledged limitations of the current implementation — randomised rather than real-market prices, file-based rather than database persistence, and single-user rather than multi-user support — are characteristic of a time-bounded academic project and have been documented transparently. A structured roadmap of seven prioritised future enhancements provides a clear path from the current prototype to a feature-complete simulation platform.

In conclusion, StockFlow constitutes a substantive and technically credible contribution to the developer's academic portfolio. It demonstrates not only the ability to apply OOP concepts in a functional system, but also the engineering discipline to design it modularly, implement it with a polished user interface, document it formally, and articulate its limitations and extension opportunities with clarity.

11. References

No.	Title / Resource	Publisher / Year	URL / ISBN
1	JetBrains Compose Multiplatform Documentation	JetBrains, 2024	https://www.jetbrains.com/compose-multiplatform/
2	Kotlin Language Documentation (v2.1)	JetBrains / Kotlin Foundation, 2024	https://kotlinlang.org/docs/home.html
3	Java SE 21 API Documentation	Oracle Corporation, 2024	https://docs.oracle.com/en/java/javase/21/docs/api/
4	Java Collections Framework Tutorial	Oracle Corporation, 2024	https://docs.oracle.com/javase/tutorial/collections/
5	ScheduledExecutorService — Java Concurrency	Oracle Corporation, 2024	https://docs.oracle.com/en/java/javase/21/docs/api/...
6	Gradle Build Tool — Kotlin DSL Documentation	Gradle Inc., 2024	https://docs.gradle.org/current/userguide/kotlin_dsl.html
7	Compose Desktop Getting Started	JetBrains, 2024	https://github.com/JetBrains/compose-multiplatform
8	Java Swing Tutorial	Oracle Corporation, 2024	https://docs.oracle.com/javase/tutorial/uiswing/
9	OOP Concepts — Java	Oracle Corporation, 2024	https://docs.oracle.com/javase/tutorial/java/concepts/
10	File I/O in Java — NIO and IO Packages	Oracle Corporation, 2024	https://docs.oracle.com/javase/tutorial/essential/io/
11	Effective Java (3rd Edition)	Joshua Bloch, Addison-Wesley, 2018	ISBN: 978-0134685991
12	GitHub Documentation — Repository Management	GitHub Inc., 2024	https://docs.github.com

12. Project File Structure

The following directory tree represents the complete structure of the StockFlow project repository.

```
StockTradingProject/
│
├── StockTradingSystem/                    # Phase 1: Java Console & Swing Version
│   ├── src/
│   │   ├── Main.java                     # Console CLI — menu-driven trading
│   │   ├── GuiMain.java                  # Swing GUI — JFrame, JTable, Timer
│   │   ├── Stock.java                    # Stock model (basic: symbol, name, price)
│   │   ├── Market.java                   # Market with 5 stocks, 2-sec simulation
│   │   └── User.java                     # User with balance and portfolio map
│   └── bin/                              # Compiled .class files
│
└── StockTradingSystemCompose/             # Phase 2: Compose Desktop Version
    ├── build.gradle.kts                   # Kotlin + Compose plugin configuration
    ├── settings.gradle.kts                # Gradle project settings
    ├── gradlew / gradlew.bat              # Gradle Wrapper scripts
    ├── run_app.sh                         # Linux convenience launcher
    ├── README.md                          # Setup and run instructions
    │
    └── src/main/
        ├── java/com/stock/backend/        # Java Backend Package
        │   ├── Stock.java                 # Extended stock model (sector,
dayHigh/Low)
        │   ├── Market.java                # 20 stocks, 6 sectors, 1-sec simulation
        │   ├── User.java                  # Full portfolio, avgPrice, P&L, watchlist
        │   ├── Transaction.java           # Immutable trade record with enum Type
        │   └── DataStore.java             # File-based persistence (~/.stockflow/)
        │
        └── kotlin/com/stock/ui/           # Kotlin UI Package
            └── Main.kt                    # Compose Desktop UI
                                           #   HeaderBar, TabRow, SectorFilter
                                           #   MarketList, WatchlistView
                                           #   TransactionHistory, TradePanel
                                           #   PortfolioPanel, SettingsView
```

13. Complete Class & Method Reference

Backend Classes — com.stock.backend

Class	Method	Parameters	Return	Description
Stock	Stock()	symbol, name, sector, initialPrice	—	4-argument constructor initialising all fields.
	getSymbol()	—	String	Returns ticker symbol.
	getName()	—	String	Returns company name.
	getSector()	—	String	Returns industry sector.
	getPrice()	—	double	Returns current price.
	getPreviousPrice()	—	double	Returns price at last tick.
	getOpenPrice()	—	double	Returns session opening price.
	getDayHigh()	—	double	Returns session high.
	getDayLow()	—	double	Returns session low.
	getChangePercent()	—	double	Returns % change from open.
	getChange()	—	double	Returns absolute $ change from open.
	updatePrice()	—	void	Applies random +/-2% price change and updates high/low.
Market	Market()	—	—	Initialises 20 stocks across 6 sectors.
	startSimulation()	—	void	Launches 1-second price update loop.
	stopSimulation()	—	void	Shuts down the executor service.
	getStocks()	—	List<Stock>	Returns all stocks.
	getStock(symbol)	String	Stock	Lookup by symbol (case-insensitive).
	getSectors()	—	List<String>	Returns distinct sector names.
	setOnUpdateCallback()	Runnable	void	Registers a tick callback.
User	User(initialBalance)	double	—	Creates fresh user.
	User(bal, init, port, avg, tx, wl)	multiple	—	Restore constructor (from saved state).
	buyStock(stock, qty)	Stock, int	boolean	Executes purchase if funds sufficient.
	sellStock(stock, qty)	Stock, int	boolean	Executes sale if shares sufficient.
	toggleWatchlist(symbo	String	void	Adds/removes symbol

Class	Method	Parameters	Return	Description
	l)			from watchlist.
	isInWatchlist(symbol)	String	boolean	Checks watchlist membership.
	getNetWorth(market)	Market	double	Computes total assets (cash + holdings).
	getTotalProfitLoss(market)	Market	double	Returns netWorth minus initialBalance.
Transaction	Transaction(type,sym,qty,price)	Type,String,int,double	—	Creates timestamped trade record.
	Transaction(type,sym,qty,price,ts)	+long	—	Restore constructor with explicit timestamp.
	getType/Symbol/Quantity...()	—	various	Standard getters for all fields.
DataStore	save(user)	User	void (static)	Serialises user state to disk.
	load(defaultBalance)	double	User (static)	Loads user from disk or creates fresh.
	reset()	—	void (static)	Deletes the save file.

UI Composables — com.stock.ui (Main.kt)

Composable Function	Purpose
main()	Application entry point. Creates Market, User, wires state and callbacks.
AppContent()	Root content composable with tabs, trade panel, and portfolio.
HeaderBar()	Displays Cash, Net Worth, and P&L stat pills.
StatPill()	Reusable coloured stat badge.
TabRow()	Navigation tabs: Market, Watchlist, History, Settings.
SectorFilter()	Horizontal scrollable sector chip row.
MarketList()	LazyColumn of all stocks with selection, prices, and watchlist stars.
WatchlistView()	Displays user-curated watchlist with prices and remove button.
TransactionHistory()	LazyColumn of all trades, newest first, with BUY/SELL badges.
TradePanel()	Stock detail, quantity input, estimated cost, Buy/Sell buttons.
PortfolioPanel()	Holdings display with total value, unrealised P&L, per-stock breakdown.
SettingsView()	Danger zone with RESET ALL DATA button.

14. Build & Run Configuration Reference

14.1 Prerequisites

Requirement	Minimum Version	Purpose
Java JDK	21 (or 17+)	Compilation target and runtime.
Git	2.x	Source code management (optional for cloning).
Gradle	Not required	Project includes Gradle Wrapper (gradlew).

14.2 Running the Application

Linux / macOS:

```
# Navigate to the Compose project directory
cd StockTradingProject/StockTradingSystemCompose

# Option 1: Using the convenience script
./run_app.sh

# Option 2: Using Gradle Wrapper directly
./gradlew run
```

Windows:

```
cd StockTradingProject\StockTradingSystemCompose
gradlew.bat run
```

14.3 Running the Java Swing Version

```
cd StockTradingProject/StockTradingSystem/src

# Compile all Java files
javac *.java -d ../bin

# Run the GUI version
java -cp ../bin GuiMain

# OR run the console version
java -cp ../bin Main
```

14.4 Data File Location

Item	Path	Description
Save Directory	~/.stockflow/	Created automatically on first trade.
User Data File	~/.stockflow/userdata.txt	Plain text, key-value format. Contains balance, portfolio, watchlist, and full transaction history.

14.5 Build Configuration Summary

Configuration	Value	Source File
Kotlin Version	2.1.0	build.gradle.kts
Compose Version	1.7.3	build.gradle.kts
JVM Toolchain	21	build.gradle.kts
Group ID	com.stock	build.gradle.kts
Main Class	com.stock.ui.MainKt	build.gradle.kts
Window Size	1400 x 900 dp	Main.kt
Default Balance	$1,000,000.00	Main.kt
Price Update Interval	1 second	Market.java
Price Fluctuation Range	+/-2% per tick	Stock.java
Number of Stocks	20	Market.java
Number of Sectors	6	Market.java